

📁 BÀI THỰC HÀNH COMPOSITE PATTERN

- ✓ Backend: ASP.NET Core Web API
 - ✓ Frontend: React (hiển thị cây Order)
 - ✓ Áp dụng Composite Pattern đúng chuẩn GoF
 - ✓ Minh họa rõ Tree Structure (Leaf + Composite)
-

🔗 BÀI TOÁN

Hệ thống **Order Management (E-commerce)** cho phép quản lý đơn hàng theo cấu trúc phân cấp.

Một Order có thể bao gồm:

- Nhóm sản phẩm (Composite)
- Sản phẩm đơn lẻ (Leaf)

👉 Hệ thống cần:

- Tính tổng số lượng sản phẩm
 - Tính tổng giá trị đơn hàng
 - Hiển thị cấu trúc dạng cây
-

🏠 KIẾN TRÚC HỆ THỐNG

```
Frontend (React)
  ↓
GET /api/orders
  ↓
OrderService
  ↓
Composite Pattern (OrderComponent)
├── OrderItem (Leaf)
└── OrderGroup (Composite)
```

💻 BACKEND – ASP.NET Core 8 Web API

1️⃣ Tạo project

```
dotnet new webapi -n OrderCompositeDemo
cd OrderCompositeDemo
```

Cấu trúc thư mục

OrderCompositeDemo



1. COMPONENT (ABSTRACTION)

Composite/OrderComponent.cs

```
public abstract class OrderComponent
{
    public string Name { get; set; }

    protected OrderComponent(string name)
    {
        Name = name;
    }

    public virtual void Add(OrderComponent component)
    {
        throw new NotImplementedException();
    }

    public virtual List<OrderComponent> GetChildren()
    {
        return new List<OrderComponent>();
    }

    public abstract int GetQuantity();
    public abstract decimal GetTotalPrice();
}
```

2. LEAF

Composite/OrderItem.cs

```
public class OrderItem : OrderComponent
{
    public int Quantity { get; set; }
    public decimal Price { get; set; }

    public OrderItem(string name, int quantity, decimal
price)
        : base(name)
    {
        Quantity = quantity;
        Price = price;
    }

    public override int GetQuantity()
    {
        return Quantity;
    }

    public override decimal GetTotalPrice()
    {
        return Quantity * Price;
    }
}
```

3. COMPOSITE

Composite/OrderGroup.cs

```
public class OrderGroup : OrderComponent
{
    private readonly List<OrderComponent> _children =
new();

    public OrderGroup(string name) : base(name) { }

    public override void Add(OrderComponent component)
    {
        _children.Add(component);
    }

    public override List<OrderComponent> GetChildren()
    {
        return _children;
    }
}
```

```
public override int GetQuantity()
{
    return _children.Sum(c => c.GetQuantity());
}

public override decimal GetTotalPrice()
{
    return _children.Sum(c => c.GetTotalPrice());
}
}
```

✿ 4. DTO MODEL

Models/OrderDto.cs

```
public class OrderDto
{
    public string Name { get; set; }
    public List<OrderDto> Children { get; set; }
}
```

✿ 5. SERVICE

Services/OrderService.cs

```
public class OrderService
{
    public OrderComponent BuildOrder()
    {
        var order = new OrderGroup("Order #1001");

        var electronics = new OrderGroup("Electronics");
        electronics.Add(new OrderItem("Laptop", 1,
1500));
        electronics.Add(new OrderItem("Phone", 2, 800));

        var accessories = new OrderGroup("Accessories");
        accessories.Add(new OrderItem("Mouse", 3, 20));

        var warranty = new OrderItem("Warranty Service",
1, 100);

        order.Add(electronics);
        order.Add(accessories);
    }
}
```

```

        order.Add(warranty);

        return order;
    }

    public OrderDto ConvertToDto(OrderComponent
component)
    {
        return new OrderDto
        {
            Name = component.Name,
            Children = component.GetChildren()
                .Select(c => ConvertToDto(c))
                .ToList();
        };
    }
}

```

❁ 6. CONTROLLER

Controllers/OrderController.cs

```

using Microsoft.AspNetCore.Mvc;
[ApiController]
[Route("api/orders")]
public class OrderController : ControllerBase
{
    private readonly OrderService _service;

    public OrderController(OrderService service)
    {
        _service = service;
    }

    [HttpGet]
    public IActionResult GetOrder()
    {
        var tree = _service.BuildOrder();
        var dto = _service.ConvertToDto(tree);

        return Ok(dto);
    }

    [HttpGet("summary")]
    public IActionResult GetSummary()
    {

```

```
        var tree = _service.BuildOrder();

        return Ok(new
        {
            totalQuantity = tree.GetQuantity(),
            totalPrice = tree.GetTotalPrice()
        });
    }
}
```

🌸 7. ĐĂNG KÝ DI

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddTransient<OrderService>();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact", policy =>
    {
        policy.WithOrigins("http://localhost:3000")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

var app = builder.Build();

app.UseCors("AllowReact");

app.UseSwagger();
app.UseSwaggerUI();

app.UseHttpsRedirection();
app.MapControllers();

app.Run();
```

▶ CHẠY BACKEND

```
dotnet run
```

Swagger:

```
https://localhost:xxxx/swagger
```

FRONTEND – REACT

Tạo project

```
npx create-react-app composite-ui  
cd composite-ui  
npm install axios
```

src/App.js

```
import React, { useEffect, useState } from "react";  
import axios from "axios";  
  
function Tree({ node }) {  
  return (  
    <ul>  
      <li>{node.name}</li>  
      {node.children &&  
        node.children.map((child, index) => (  
          <Tree key={index} node={child} />  
        ))}  
    </ul>  
  );  
}  
  
function App() {  
  const [order, setOrder] = useState(null);  
  
  useEffect(() => {  
    axios.get("https://localhost:xxxx/api/orders")  
      .then(res => setOrder(res.data));  
  }, []);  
  
  return (  
    <div style={{ padding: 40 }}>  
      <h2>Order Tree</h2>  
      {order && <Tree node={order} />}  
    </div>  
  );  
}
```

```
}  
  
export default App;
```

▶ CHẠY FRONTEND

```
npm start
```

🔗 LUỒNG XỬ LÝ

Khi gọi `/api/orders`

- Build cây Order
 - Convert → DTO
 - Trả JSON
-

Khi gọi `/api/orders/summary`

```
tree.GetQuantity();  
tree.GetTotalPrice();
```

🔗 Composite xử lý:

- Leaf → trả trực tiếp
 - Composite → cộng từ children
-

🔗 KIẾN THỨC ĐẠT ĐƯỢC

✓ Hiểu Composite Pattern (GoF)

✓ Phân biệt:

- Leaf (OrderItem)
- Composite (OrderGroup)

✓ Áp dụng thực tế:

- Order / Cart
- Invoice
- Product hierarchy

BÀI TẬP MỞ RỘNG

1. Thêm:

- Discount theo nhóm
- Tax calculation

2. CRUD:

- Thêm / sửa / xóa item

3. Áp dụng:

- Shopping Cart
- Invoice System